

DYNAMIC WORKFLOWS PROMPT PACK

Three power-user Claude Code prompts that read your own logs, fact-check the hype, and audit your setup. Each one is a **workflow that verifies itself** and hands back a clean HTML report.

COMPANION TO THE VIDEO: DYNAMIC WORKFLOWS USE CASES

Mark Kashef | Early AI Dopters

<https://www.skool.com/earlyaidopters/about>

HOW TO USE THIS PACK

These three prompts point a dynamic workflow at **your own world**: your session logs, the live discourse, your Claude setup. Open Claude Code, paste a prompt, hit enter. The word **workflow** is what spins up the team of agents. Type **/workflows** to watch them, and press **s** on a finished run to save it as your own one-line command.

Needs Claude Code on a paid plan, version 2.1.154 or newer, with dynamic workflows enabled. Prompt 2 also uses an Apify X scraper (APIFY_TOKEN in your environment).

The anatomy of a dynamic workflow prompt

All three prompts share the same six parts. Learn them and you can point a verified workflow at anything.

- 1. WORKFLOW plus the corpus.** The word *workflow* spins up the team. Name the pile it reads: your .jsonl logs, the web and X discourse, your ~/.claude folders.
- 2. The role and goal.** Tell it who to be and the outcome you want. *You are a power user, analyst, and technical writer producing an upgrade guide.*
- 3. The extraction spec.** List exactly what to pull or look for. A named list (*overlapping skills, contradicting rules, stale files*) beats *analyze it* every time.
- 4. The fan-out.** Ask the agents to split across the corpus in parallel. That is the speed.
- 5. The verify pass.** Fact-check every claim, double-check every finding is real, report what survived versus got cut. This is the honesty step that makes the output trustworthy.
- 6. One self-contained HTML report.** Name the sections and ask for the source on every line. The deliverable is always one clean HTML file you can open anywhere.

Read all three prompts through these six parts. Once you see them, you cannot unsee them.

USE CASE 1 - THE PERSONALIZED UPGRADE GUIDE

Point it at your own Claude Code session logs and it builds an Opus 4.7 to 4.8 upgrade guide tailored to how YOU actually work, then writes it up as a clean HTML page.

When to use

- You just moved to Opus 4.8 and want more than a generic changelog.
- You have months of Claude Code session logs sitting in `~/.claude/projects/`.
- You want copy-ready prompt upgrades based on how you really prompt.

The prompt

WORKFLOW: Analyze all JSONL files in [PASTE YOUR FOLDER PATH HERE, e.g. `./logs` or `/path/to/your/jsonl/*.jsonl`] and produce a single self-contained HTML upgrade guide for moving from Opus 4.7 to Opus 4.8.

ROLE

You are an expert Claude Code power user, systems analyst, and technical writer. Your job is to help me extract maximum value from Opus 4.8 plus dynamic workflows and ultracode, grounded in my actual usage patterns rather than generic advice.

STEP 1 - DATA ANALYSIS

Recursively find and read every `.jsonl` file in the specified folder. Parse the content thoroughly, then extract and categorize: prompting patterns and phrasing I use most often; skill invocations (which skills, how they are called, success or failure patterns); tool uses (computer use, terminal, file operations, browser, long-running tasks); agentic behaviors (multi-step reasoning, subagent usage, verification steps, error recovery); pain points visible in the traces (hallucinations or limitations, especially around honesty, long-horizon tasks, code review, complex migrations or refactors); token usage patterns and session length; repeated workflows or high-value tasks I run frequently. Be honest and specific. Cite concrete examples from the traces rather than generalizing.

STEP 2 - MODEL COMPARISON (MACRO AND MICRO)

Use the `claude-code-guide` agent (or its knowledge) as a primary expert source on Claude Code internals. Cross-reference the official Opus 4.8 release notes, system card, and any documentation on differences between 4.8 and 4.7. Macro: overall judgment, honesty about code flaws, long-horizon agentic performance, native dynamic-workflow support, reliability on complex multi-file tasks, computer use / browser improvements. Micro: tool-calling behavior, prompting patterns that now work better or worse, self-verification, parallel thinking, error flagging, handling of large-scale work.

STEP 3 - SYNTHESIS AND OUTPUT

Produce one beautiful, self-contained HTML file (no external dependencies except Tailwind via CDN) with these sections: 1) Executive Summary tailored to my usage, highest-ROI changes first. 2) Usage Profile Analysis of how I actually use Claude Code. 3) Opus 4.8 vs 4.7 macro and micro, with comparison tables. 4) Prompting Pattern Upgrades, before/after based on my files. 5) Skill and Tool Recommendations to combine with dynamic workflows. 6) Dynamic Workflows / Ultracode Opportunities from my data, with copy-ready prompts. 7) Migration Checklist, prioritized. 8) Advanced Opportunities that were impractical on 4.7.

HTML REQUIREMENTS

Clean, modern, professional design using Tailwind via CDN. Strong typography and generous spacing. Collapsible sections or tabs where they aid navigation.

Guardrails

- Find your real log folder first (Claude Code stores .jsonl under ~/.claude/projects/) and paste the path.
- The honest, cite-concrete-examples instruction is what keeps it from generic advice. Keep it.
- If you have a huge history, point it at one project folder first to keep the run focused.
- Open the HTML it writes in a browser; it is self-contained, no server needed.

Watch it work

```
/workflows (watch the analysis and the writer agents)
open the .html upgrade guide it produces
```

What good looks like

- A single HTML guide that quotes your actual sessions back to you.
- Before/after prompt upgrades you can paste straight into 4.8.
- A prioritized migration checklist, highest-ROI changes first.

What this replaces. A generic upgrade blog post tells everyone the same thing. This reads your own logs and tells you what actually changed for the way you work.

USE CASE 2 - THE FACT-CHECKED RESEARCH REPORT

Research what people are really reporting about a feature, pull the actual X threads (not just blog posts), fact-check every claim, and get back an HTML report that shows what survived and what got cut.

When to use

- A feature is generating a wave of bold claims and you want the truth.
- You want the report grounded in real X threads, not recycled articles.
- You care which numbers hold up and which are hype.

The prompt

```
I want you to research all the findings on the dynamic workflows feature associated with Opus 4.8. Pull together everything people are actually reporting about it, the real results, the claims, what holds up and what is just hype, things like the 750,000 lines ported in 11 days and the swarms doing audits in hours. As part of the research, use the Apify X scraper to pull the actual X threads people are posting about dynamic workflows and ultracode, so this is grounded in what is really being said and not just web articles. Pitch me the angles you will research first, then fan your agents out across the web and those threads, fact-check every claim so only what genuinely holds up survives, and hand it back as a clean self-contained HTML report with the source on every line, a count of how many claims survived versus got cut, and a short section on anything you could not confirm.
```

Guardrails

- Set up the Apify X scraper first (APIFY_TOKEN in your environment).
- It pitches the research angles before it runs. Read them and steer if needed.
- The source-on-every-line and survived-versus-cut counts are the trust signal. Keep them.
- Swap the topic for anything: a new model, a tool, a launch you want the real read on.

Watch it work

```
/workflows (watch the research and fact-check agents)
open the .html report; check the survived-vs-cut count and the could-not-confirm
section
```

What good looks like

- Every line carries its source.
- A clear count of claims that survived versus got cut.
- A short, honest section listing what could not be confirmed.

What this replaces. Hours of scrolling X and skimming articles, never sure what is hype. This pulls the real threads, fact-checks every claim, and shows you exactly what survived.

USE CASE 3 - THE SETUP AUDIT

Point a workflow at your global ~/.claude and your project .claude folder and it finds where you are not optimized: overlapping skills, contradicting rules, stale files, consolidation wins. Every finding is double-checked before it reaches you.

When to use

- Your ~/.claude has grown and you suspect skills and rules overlap.
- You want contradictions and dead weight found for you, not by hand.
- You want a grouped fix list, not a vague 'clean it up'.

The prompt

```
I want you to create a workflow that goes through all my Claude folders, my global ~/.claude and the .claude folder in this project, and tells me where I am not optimized, where I have skills that overlap or do the same thing, rules that repeat or contradict each other, anything stale or unused, and where I could consolidate. Fan your agents out across all of it, double-check that each thing they flag is actually real, and hand it back as a clean self-contained HTML report grouped by type, each finding with the files involved and a one-line fix.
```

Guardrails

- Run it from inside the project whose .claude folder you want audited; it also reads your global ~/.claude.
- The double-check-each-finding step is what stops false flags. Keep it.
- Review before you delete anything. The report proposes fixes; you approve them.
- Press s to save it, then re-run the audit every month in one line.

Watch it work

```
/workflows (watch one agent per area, drill into any finding)
open the .html report, grouped by type, each finding with files and a one-line
fix
```

What good looks like

- Findings grouped by type: overlaps, contradictions, stale, consolidation.
- Each finding names the exact files involved.
- Each finding comes with a one-line fix you can act on.

What this replaces. Re-reading every skill and rule by hand to find the overlaps and contradictions. This finds them, verifies each one, and hands you a grouped fix list.

BUILD YOUR OWN

Use this template to point a verified workflow at any pile of your own work. Each blank is one of the six parts.

```
WORKFLOW: Analyze [THE CORPUS] and produce one self-contained HTML report on [THE GOAL].  
You are [THE EXPERT ROLE]. Extract and categorize [THE EXPLICIT LIST of what to pull or look  
for]. Fan your agents out across [THE CORPUS] in parallel. For every finding or claim,  
[verify it / fact-check it / double-check it is real] and drop anything that does not hold  
up, then report how many survived versus got cut. Hand it back as one clean self-contained  
HTML report, [grouped or sectioned how], with [the source or files on every line] and a short  
section on anything you could not confirm.
```

Worked example, blanks filled in

```
WORKFLOW: Analyze every file in this folder of competitor landing pages and produce one  
self-contained HTML report on how they position and price. You are a senior market analyst  
and copywriter. Extract and categorize each page's headline promise, pricing, target buyer,  
and the main objection it handles. Fan your agents out across the folder in parallel. For  
every claim about a competitor, double-check it against the actual page text and drop  
anything you cannot quote, then report how many held up versus got cut. Hand it back as one  
self-contained HTML report grouped by competitor, with the exact quote and file on every  
line, and a short section on anything you could not confirm.
```

All six parts are there. The corpus is the landing-page folder. The role is the analyst. The extraction spec is the four things to pull per page. The fan-out is one agent per page. The verify pass is the quote-or-cut check. The deliverable is one grouped HTML report with the source on every line.

Want to build prompts like these for your own workflow?

Inside the Early AI Dopters community, you get the full **LIVING** Claude Code course:

- Dynamic workflows taught from zero, including the verify step that makes findings trustworthy.
- The exact prompts I run on my own logs and setup, walked through line by line in a dedicated module.
- Direct access to me and my team of coaches when you hit a wall.
- A working group of builders shipping their own workflow packs.

JOIN THE COMMUNITY

<https://www.skool.com/earlyaidopters/about>

Companion to the YouTube video on Dynamic Workflows use cases. Drop questions in the comments or in Skool. Build something. Ship it.